Software Engineering

Chapter 8 — Software Testing

Complete Study Notes

Based on Sommerville's Software Engineering, 10th Edition

#	Topic	Slides
1	Component Testing	1
2	Interface Testing	2 – 7
3	System Testing	8 – 13
4	Test-Driven Development (TDD)	14 – 18
5	Regression Testing	19
6	Release Testing	20 – 26
7	User Testing	27 – 32

Topic 1: Component Testing (Slide 1)

When you build software, you first test each small piece individually — that's unit testing. But once those pieces are done, you connect them together into a bigger component. Component testing is simply asking: Now that these pieces are connected, do they work together correctly through their shared interface?

Real Example — Uber App

Imagine Uber has these individual objects inside its Booking Component:

```
[Location Object] + [Driver Matching Object] + [Fare Calculator Object]
                          ↓
                = Booking Component
```

Unit testing already verified each object works alone. But component testing now asks: when a user books a ride, do all three objects talk to each other correctly through the component's interface? Does the location get correctly passed to driver matching? Does the fare calculator receive the correct distance?

Level	What it tests	Example
Unit Testing	Each object in isolation	Does fare calculator compute correctly?
Component Testing	Objects working together via interface	Does location feed correctly into fare calc?

■ Big Takeaway: Just because every individual part works perfectly alone doesn't mean they'll work correctly together. Component testing catches the bugs that only appear when objects start talking to each other — and it tests through the component's defined interface, not by poking inside each object individually.

Topic 2: Interface Testing (Slides 2 – 7)

When two components talk to each other, they communicate through an interface — think of it like a plug and socket. Interface testing checks that this communication works correctly. These errors can NEVER be caught by unit testing because they only appear when two components actually interact.

Real Example — Daraz

```
[Search] → [Inventory] → [Payment] → [Delivery]
Each arrow = an interface. Interface testing checks every arrow.
```

The 4 Types of Interfaces

1. Parameter Interface

One component passes data to another through method parameters — the most common type.

```
inventory.checkAvailability(productId, quantity);
//                          ↑ these are parameters being passed
```

Bug example: Search component passes parameters in wrong order — productId and quantity swapped — causing wrong product to be checked.

2. Shared Memory Interface

Two components share the same block of memory — one writes, the other reads. Classic in embedded/IoT systems.

```
[Sensor Component]   → writes temperature to shared memory
[Analyser Component] → reads temperature from shared memory
```

Bug example: Analyser reads before Sensor has updated — gets stale data. In a hospital monitoring system, this could display dangerously outdated vitals.

3. Procedural Interface

One component calls a set of procedures that belong to another component.

```
[Order Component] calls → payment.processCard()
                          payment.generateReceipt()
                          payment.sendConfirmation()
```

4. Message Passing Interface

Components request services from each other by sending messages — common in client-server systems like REST APIs.

```
[App] → sends HTTP message → [API Server]
      ← receives response  ←
```

The 3 Types of Interface Errors

Error Type	What it means	Real Example
Interface Misuse	Parameters passed in wrong order or wrong type	calculateShipping(distance, weight) instead of (weight, distanc
Interface Misunderstanding	Wrong assumption about how the other component works	Calling binary search with an unsorted array
Timing Error	Reading data before it has been updated	Hospital monitor reads old vitals before sensor has written new

Why Interface Testing is Hard

• Overflow assumption: Component A treats a queue as infinite, Component B's queue is actually finite — overflow crash that neither component causes alone.

• Fault propagation: Wrong value in Component A → processed wrongly by B → crash in C. The bug is in A but the error appears in C.

```
[A passes wrong value] → [B processes it, looks fine] → [C crashes]
Bug is in A but appears in C — very hard to trace!
```

Interface Testing Guidelines

Guideline	What it means	Real Example
Test extreme ranges	Pass min and max values	Array with 0 items AND with 1 million items
Design tests that cause failure	Deliberately try to break the interface	Pass null, negative numbers, empty strings
Use stress testing	Flood with messages for timing errors	Send 10,000 simultaneous requests to payment API

Interface Testing is BLACK-BOX testing — you sit outside the components and only check: I send this data through the interface, do I get the correct response back? You don't look at the internal code of either component.

■ Big Takeaway: Unit testing checks if each component works alone. Interface testing checks if they work together. The most dangerous bugs hide at the boundaries between components — not inside them.

Topic 3: System Testing (Slides 8 – 13)

System testing takes ALL components together as one complete system and tests it as a whole. It's black-box testing — you don't care about internal code anymore. You just check: does the complete system behave the way it's supposed to?

> Think of it like building a car. Unit testing checked each part. Component testing checked if engine + transmission work together. System testing is taking the fully assembled car out for a test drive.

What System Testing Specifically Checks

Check	What it means	HBL Banking App Example
Compatible components	All parts speak the same language	Login passes correct user object to Dashboard
Interact correctly	Right sequence of calls	Transfer happens BEFORE notification is sent
Right data at right time	Correct data flows across interfaces	Rs.5000 amount reaches Transaction History accurately

System Testing in Real Companies

- Mixed components: Real systems combine newly written code, reusable components from previous projects, and off-the-shelf systems (like payment gateways).
- Multiple teams: Different teams build different components — system testing is where their work meets for the first time.
- Separate testing team: In large companies, system testing is done by a dedicated QA team with no involvement from developers — fresh eyes catch what developers miss.

Use-Case Based System Testing

Since system testing is about interactions, the best way to design test cases is from use cases. A use case naturally forces multiple components to interact — which is exactly what you want to test.

Example — Foodpanda Order Flow

```
Customer → [App UI] → [Order Service] → [Restaurant Service]
                                  ↓
                      [Payment Service] → [Delivery Service]
                                                  ↓
                                        [Notification] → Customer
```

Test Case	What it tests
TC1	Place order → confirm received → restaurant notified
TC2	Payment processed → correct amount deducted → receipt generated
TC3	Order delivered → customer notified → order marked complete
TC4	Restaurant rejects → customer notified → refund initiated

Testing Policies

Exhaustive system testing is impossible — the number of possible combinations is practically infinite. So companies define testing policies: agreed rules about what MUST be tested.

Policy	Real Meaning
All menu functions must be tested	Every button/feature a user can click must be tested at least once
All inputs tested with correct AND incorrect data	Don't just test happy paths — test what happens with invalid inputs

■ Big Takeaway: System testing is the first time the complete software is tested as one unit. It catches compatibility problems, wrong data flows, and broken sequences that unit and interface testing can never find. The best approach is use-case based testing because use cases naturally force all components to interact.

Topic 4: Test-Driven Development (TDD) (Slides 14 – 18)

In normal development, developers write code first and test it later. TDD completely flips this: Write the test FIRST. Then write the code to pass that test. The test becomes your target — you write code until you hit that target, then move to the next feature.

Normal Development vs TDD

Normal Development	Test-Driven Development
1. Write code	1. Write test FIRST
2. Manually test it	2. Run test → it FAILS (expected!)
3. Find bugs	3. Write minimum code to pass
4. Fix bugs	4. Run test → it PASSES ■
5. Hope nothing else broke	5. Refactor code, move to next feature

The TDD Cycle — Red → Green → Refactor

Phase	Color	Meaning
Write test, run it, it fails	■ Red	Expected! Code doesn't exist yet
Write code, run test, it passes	■ Green	Code satisfies the test
Clean up the code	■ Refactor	Make it neat without breaking the test

Concrete Example — University Portal Login

Requirement: Reject login if password is less than 8 characters

Step 1 — Write the test FIRST:

```
@Test
public void testPasswordTooShort() {
    LoginService login = new LoginService();
    String result = login.authenticate('ali@uok.edu', 'abc');
    assertEquals('Password too short', result);
}
```

Step 2 — Run the test: FAIL (LoginService doesn't exist yet) — this is expected and good!

Step 3 — Write minimum code to pass:

```
public String authenticate(String email, String password) {
    if (password.length() < 8)
        return 'Password too short';
    return 'Login successful';
}
```

Step 4 — Run test again: PASS ■ — now move to the next requirement.

4 Benefits of TDD

Benefit	What it means	Real Impact
Code Coverage	Every line of code has a test behind it	No untested code — defects discovered early
Regression Testing	All previous tests run automatically after every change	New features cannot silently break old ones
Simplified Debugging	Failing test points directly at the problem	No need to search through thousands of lines
System Documentation	Tests describe what code is supposed to do	New developers understand the system by reading tests

TDD Works in Both Agile and Plan-Based Projects

Project Type	How TDD fits
Agile	Natural fit — short sprints, write test → code → refactor each sprint
Plan-based (Waterfall)	Requirements defined upfront so tests can be written from specs before coding

■ Big Takeaway: TDD reverses the normal development process — tests come first, code comes second. This forces developers to think clearly about what their code should do BEFORE writing it, guarantees every line is tested, catches bugs instantly, and produces tests that serve as living documentation. It's not just testing — it's a whole different way of writing software.

Topic 5: Regression Testing (Slide 19)

Every time you change code — fixing a bug, adding a feature, improving performance — you risk accidentally breaking something that was already working. Regression testing is re-running all previous tests after every change to make sure nothing got broken. The name 'regression' means going backwards — you're checking that software hasn't regressed to a broken state.

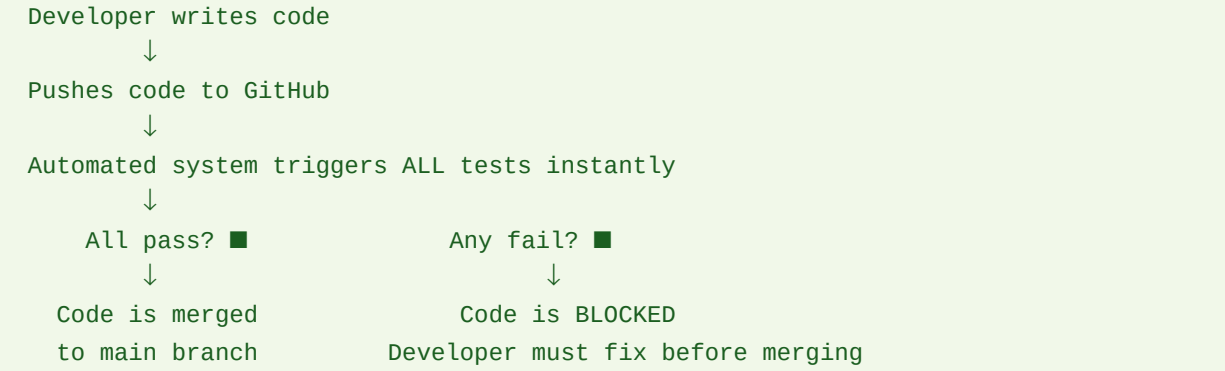
Real Example — Foodpanda Adding 'Schedule Order' Feature

Without Regression Testing	With Regression Testing
■ Schedule Order works	■ Schedule Order works
■ Cancel Order button breaks — nobody notices	■ Cancel Order test FAILS instantly
■ Emails stop sending — nobody notices	■ Email confirmation test FAILS instantly
■ Customers complain weeks later	■ Developer fixes both before release

Manual vs Automated Regression Testing

	Manual	Automated
How	Tester re-tests everything by hand	Tests run automatically with one command
Cost	Extremely expensive — takes days	Almost free — takes minutes
Human error	Testers forget to check some things	Nothing missed — all tests run every time
Frequency	Done rarely because it is painful	Done after EVERY single code change

How It Works in Practice — CI/CD Pipeline



TDD dramatically reduces regression testing costs because all tests are already written as part of development. Every time you add code in TDD, the regression test suite grows automatically.

■ Big Takeaway: Software is never truly done — it's constantly changing. Every change is a risk. Regression testing is your safety net. Manually it's too expensive to do properly, but with automation it costs almost nothing and runs after every single change.

Topic 6: Release Testing (Slides 20 – 26)

Release testing is done on a version of the software about to be shipped to the outside world. The goal is not to find every bug — it's to build enough confidence to say: This software is good enough to release. It is a black-box, validation-focused process where test cases are derived from the system specification.

Release Testing vs System Testing

	System Testing	Release Testing
Goal	Find and kill bugs	Confirm software is good enough to ship
Mindset	"Let's break it"	"Let's validate it meets requirements"
Done by	Development / QA team	Separate release testing team
Type	Defect testing	Validation testing
Analogy	Doctor looking for diseases	Health certificate check before travel

Type 1 — Requirements-Based Testing

Every single requirement written in the specification must have at least one test case. You go through requirements one by one and ask: Does the system actually do what we promised?

Case Study — Dvago Online Medicine Store

Requirement	Test Case
Customer searches for medicine availability	Search Panadol → shows available / Search XYZ → shows not found
Patient uploads prescription before adding to cart	Try adding prescription medicine WITHOUT uploading → system blocks it
If in prescription list → stock reduced and shipped	Upload valid prescription → add to cart → verify stock decreases by 1
If out of stock → error shown	Search medicine with 0 stock → Item Sold Out message appears

Type 2 — Performance Testing

Performance testing checks that the system works correctly under real-world load conditions. Your slide example: a system that handles 300 transactions at a time.

```
50 users  → ■ System responds normally
100 users → ■ Still fine
200 users → ■ Slight slowdown
300 users → ■■  At designed limit
400 users → ■ System starts degrading
500 users → ■ System fails
```

The goal is finding exactly where the system starts breaking down — and ensuring that point is well beyond normal expected usage.

Type 3 — Stress Testing

Stress testing deliberately overloads the system beyond its limits to see HOW it fails — not just IF it fails.

Failure Type	What it means	Real Example
Fail-hard	System crashes, data lost, users lose everything	Bank crashes mid-transaction — money deducted but not transfe
Fail-soft	System slows down gracefully, no data corruption	Netflix buffers and reduces quality instead of crashing

> Real Pakistan Example — FBR Tax Portal: Every year before the deadline, the FBR portal crashes because millions file simultaneously. Proper stress testing would have identified the breaking point before launch.

The Two Key Things Stress Testing Identifies

- The degradation point — where performance starts getting worse
- The failure point — where the system actually breaks

A well-engineered system has a large gap between these two points — it degrades slowly and gracefully before ever actually failing.

> **■ Big Takeaway: Release testing is the final quality gate before software reaches real users. Requirements-based testing ensures every promise was kept. Performance testing ensures the system handles real load. Stress testing ensures that when the system fails, it fails safely without corrupting data.**

Topic 7: User Testing (Slides 27 – 32)

User testing is when real users — not developers or testers — formally test the system. Even when comprehensive system and release testing have been carried out, user testing remains essential because the user's real working environment has a major effect on reliability, performance, usability and robustness that simply cannot be replicated in a testing lab.

> Developers test software in ideal, controlled conditions. Real users use it in unpredictable, messy real-world conditions — slow internet, old phones, unusual workflows. Only real users can reveal these problems.

The 3 Types of User Testing

1. Alpha Testing

Users work closely with the development team and test the software at the developer's site. This is the first time real users interact with the software — providing direct feedback to help developers build an effective interface.

Who does it	Where	Goal	Real Example
Selected users from target audience	Developer's office/site	Find usability issues, interface problems, missing features	HBL invites 20 employees to test new app

2. Beta Testing

A release of the software is made available to a wider group of users — early adopters — to experiment with and report problems they discover. The company does not sit with these users; feedback comes in remotely.

Who does it	Where	Goal	Real Example
Early adopters, volunteer users	User's own environment	Discover bugs in real-world conditions across diverse environments	Firms release beta version to 1000 users

3. Acceptance Testing

Customers formally test the system to decide whether it is ready to be accepted from the developers and deployed in their environment. This is the final go/no-go decision before the software is officially handed over.

Who does it	Where	Goal	Real Example
The actual client/customer	Customer's real environment	Decide: is this system ready for deployment?	NADRA tests new passport system before offici

The 6 Stages of the Acceptance Testing Process

Your slide shows a clear 6-stage process (Figure 8.11):

Stage	What happens	Output
1. Define acceptance criteria	Agree on what "acceptable" means — unclear requirements addressed	Test criteria defined
2. Plan acceptance testing	Decide resources, time, budget, risk mitigation	Test plan

Stage	What happens	Output
3. Derive acceptance tests	Create tests for functional AND non-functional requirements	Test cases
4. Run acceptance tests	Tests executed in actual working environment	Test results
5. Negotiate test results	Almost impossible all tests pass — developer and customer negotiate	Testing report
6. Accept or reject system	If acceptable → deployed. If not → revision required	Final decision

Alpha vs Beta vs Acceptance — Quick Comparison

	Alpha	Beta	Acceptance
Who tests	Selected users at developer site	Early adopters in their own environment	The actual paying customer
Location	Developer's site	User's own environment	Customer's real environment
Feedback style	Direct — face to face with team	Remote — reports sent in	Formal — contractual decision
Goal	Improve interface design	Find real-world bugs	Go/No-Go deployment decision
Analogy	Test drive at dealership	Borrow a car for a week	Sign the purchase contract

Why User Testing Cannot Be Skipped

What developers test for	What only users can reveal
Does it work correctly?	Is it actually usable by non-technical people?
Does it handle edge cases?	Does it work on slow 3G connections?
Does it perform under load?	Does it work on old Android phones?
Are requirements met?	Does it match how people actually work?

■ **Big Takeaway: User testing is the final reality check. Developers test software in ideal conditions but real users use it in messy, unpredictable real-world conditions. Alpha testing refines the interface, beta testing catches real-world bugs across diverse environments, and acceptance testing is the customer's formal go/no-go decision. No amount of developer testing can replace the insight of real users in their real environment.**

Complete Chapter Summary — Testing Levels

Testing Level	What it tests	Who does it	Type
Unit Testing	Individual functions/methods in isolation	Developer	White-box
Component Testing	Multiple objects working together via interface	Developer	Black-box
Interface Testing	Communication between components	Developer / QA	Black-box
System Testing	Complete integrated system end-to-end	Separate QA team	Black-box
Release Testing	System ready for external use	Release team	Black-box
User Testing	System in real-world user conditions	Real users / customers	Black-box

Testing Type	When used	Key Technique
White-box / Structural	When you need to test internal code paths	Basis Path Testing, CFG, Cyclomatic Complexity
Black-box / Behavioral	When you test behavior from the outside	Interface, System, Release, User Testing
TDD	Throughout development — test before code	Red-Green-Refactor cycle
Regression Testing	After every code change	Automated test suite re-run
Stress Testing	Before release under performance requirements	Gradually increase load until failure

Sommerville's Software Engineering, 10th Edition — Chapter 8